```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
cancer = load_breast_cancer()

X = pd.DataFrame(
    cancer.data,
    columns=cancer.feature_names
)

y = pd.Series(cancer.target)

print(X.shape)
X.head()
```

```
(569, 30)
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	c
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	

5 rows × 30 columns

```
print("Features:", X.shape[1])
```

```
print("Samples:", X.shape[0])

print("\nTarget Classes:")
print(pd.Series(y).value_counts())
```

```
Features: 30
Samples: 569

Target Classes:
1    357
0    212
Name: count, dtype: int64
```

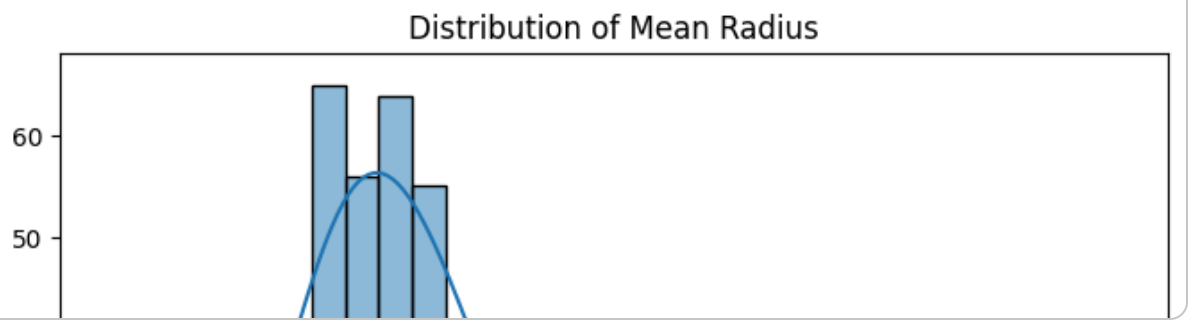
```
X.isnull().sum().sum()
```

```
np.int64(0)
```

```
plt.figure(figsize=(8,5))

sns.histplot(
    X["mean radius"],
    bins=30,
    kde=True
)

plt.title("Distribution of Mean Radius")
plt.show()
```




```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print(X_train.shape)
print(X_test.shape)
```

```
(455, 30)
(114, 30)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
y_pred = knn.predict(X_test_scaled)
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.956140350877193
```

```
precision = precision_score(y_test, y_pred)
print("Precision:", precision)
```

```
Precision: 0.958904109589041
```

```
recall = recall_score(y_test, y_pred)
print("Recall:", recall)
```

```
Recall: 0.9722222222222222
```

```
f1 = f1_score(y_test, y_pred)
print("F1 Score:", f1)
```

```
F1 Score: 0.9655172413793104
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.95      0.93      0.94        42
           1       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

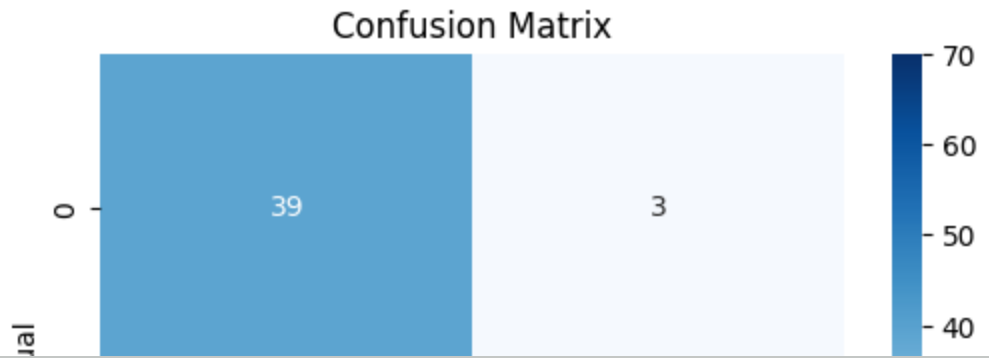
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```




```
k_values = [1, 3, 5, 7, 9, 11]

accuracy_scores = []

for k in k_values:

    model = KNeighborsClassifier(n_neighbors=k)

    scores = cross_val_score(
        model,
        X_train_scaled,
        y_train,
        cv=5,
        scoring='accuracy'
    )

    accuracy_scores.append(scores.mean())
```

```
print(accuracy_scores)
```

```
[np.float64(0.945054945054945), np.float64(0.9692307692307693), np.float6
```

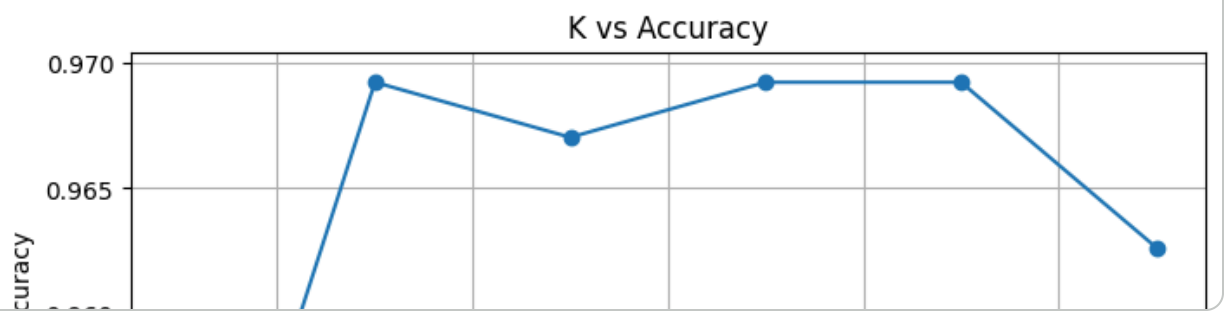
```
plt.figure(figsize=(8,5))

plt.plot(
    k_values,
    accuracy_scores,
    marker='o'
)

plt.xlabel("K Value")
plt.ylabel("Cross Validation Accuracy")
plt.title("K vs Accuracy")

plt.grid(True)

plt.show()
```




```
best_k = k_values[np.argmax(accuracy_scores)]

print("Best K:", best_k)
```

```
Best K: 3
```

```
final_knn = KNeighborsClassifier(
    n_neighbors=best_k
)

final_knn.fit(
    X_train_scaled,
    y_train
)

final_pred = final_knn.predict(
    X_test_scaled
)
```

```
print("Accuracy:",
```

```
      accuracy_score(y_test, final_pred))

print("Precision:",
      precision_score(y_test, final_pred))

print("Recall:",
      recall_score(y_test, final_pred))

print("F1 Score:",
      f1_score(y_test, final_pred))
```

```
Accuracy: 0.9824561403508771
Precision: 0.972972972972973
Recall: 1.0
F1 Score: 0.9863013698630136
```

```
sample = X.iloc[[0]]

sample_scaled = scaler.transform(sample)

prediction = final_knn.predict(
    sample_scaled
)

print("Prediction:", prediction[0])
```

```
Prediction: 0
```

```
print("""
KNN Classification Project Summary

1. Loaded Breast Cancer Dataset.
2. Split data into training and testing sets.
3. Applied StandardScaler.
4. Trained KNN classifier.
5. Evaluated using Accuracy, Precision,
   Recall, F1-Score and Confusion Matrix.
6. Tuned K values using Cross Validation.
7. Selected optimal K and retrained model.
8. Tested prediction on new sample data.
""")
```

```
KNN Classification Project Summary

1. Loaded Breast Cancer Dataset.
2. Split data into training and testing sets.
3. Applied StandardScaler.
4. Trained KNN classifier.
5. Evaluated using Accuracy, Precision,
   Recall, F1-Score and Confusion Matrix.
6. Tuned K values using Cross Validation.
7. Selected optimal K and retrained model.
8. Tested prediction on new sample data.
```